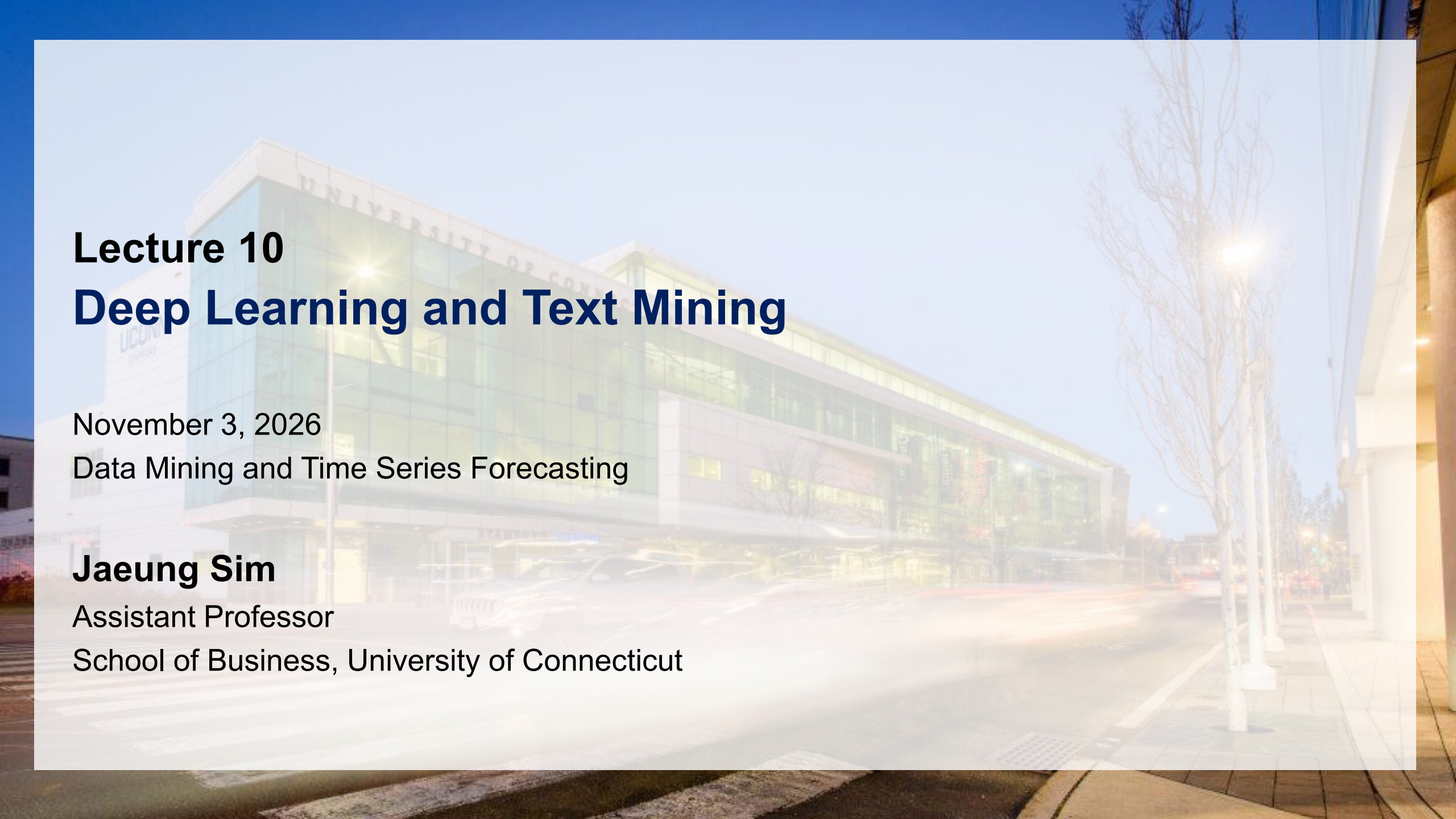


Lecture 10

Deep Learning and Text Mining

November 3, 2026

Data Mining and Time Series Forecasting

Jaeung Sim

Assistant Professor

School of Business, University of Connecticut

Agenda

- **Take-home Midterm Feedback**
- **Deep Learning and Text Mining**
  - Artificial Neural Networks
  - RNN-based Models
  - Embedding and the Transformer
- **FYI: Optional Python Hands-on**
- **Announcements**



Take-home Midterm Feedback

Overall Statistics and Question Review

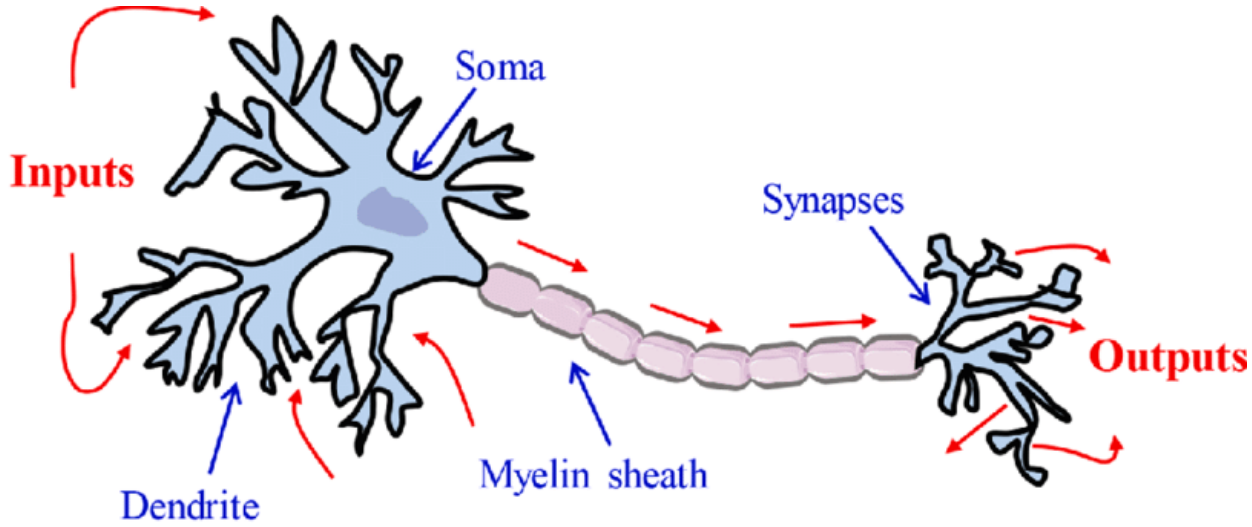


Deep Learning and Text Mining

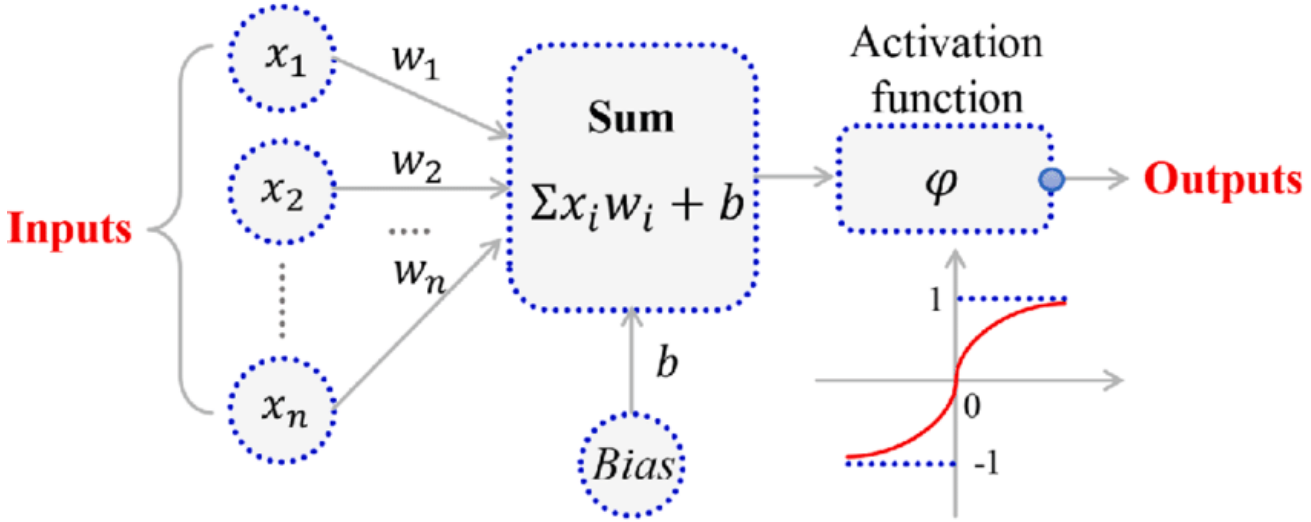
About model assessment and selection

Neural Network

- What is a **Neural Network**?
  - A neural network is a type of machine learning algorithm that is modeled after the structure and function of the human brain.
  - It is composed of interconnected **nodes** or artificial **neurons** that are organized in layers.



(a) Biological neuron



(b) Artificial neuron

Neural Network

- What is **Deep Learning**?
  - Deep learning uses artificial neural networks with **many layers** (thus, "**deep**") to model and solve complex problems.
  - It is based on the idea of learning hierarchical representations of data, where each layer of the network learns to represent increasingly abstract features of the input data.
  - Deep learning algorithms require large amounts of data and computational power.

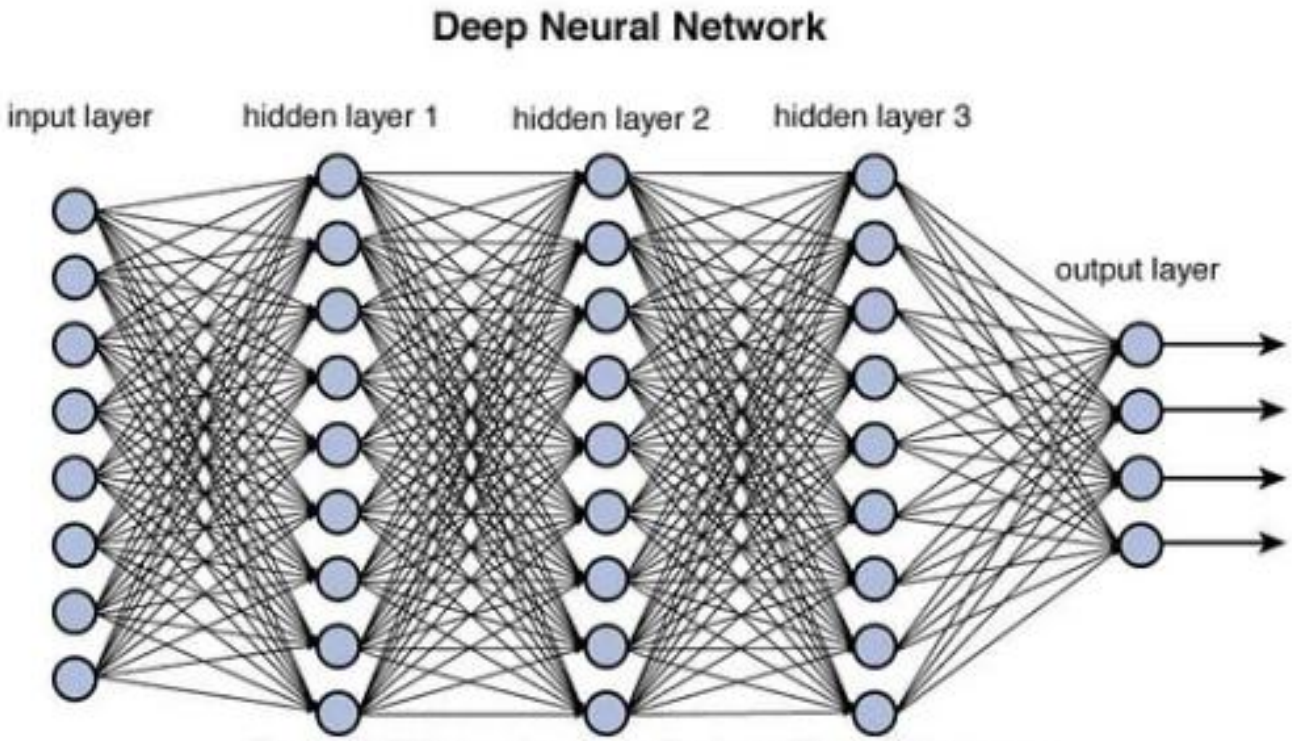
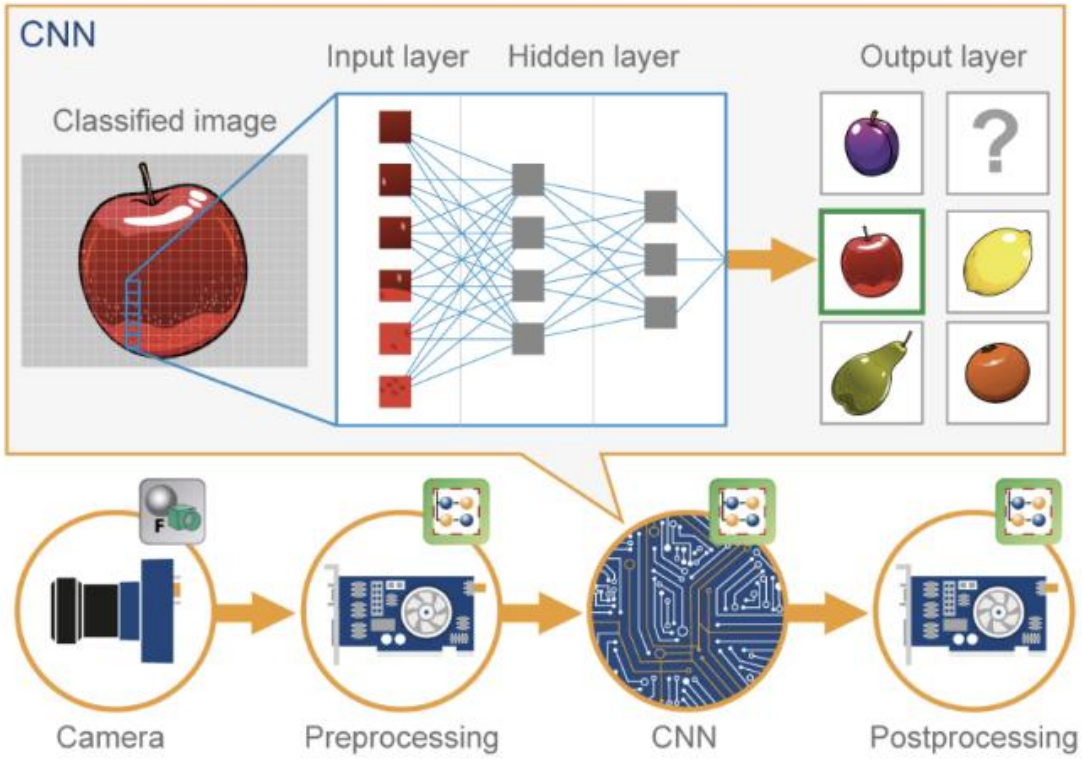
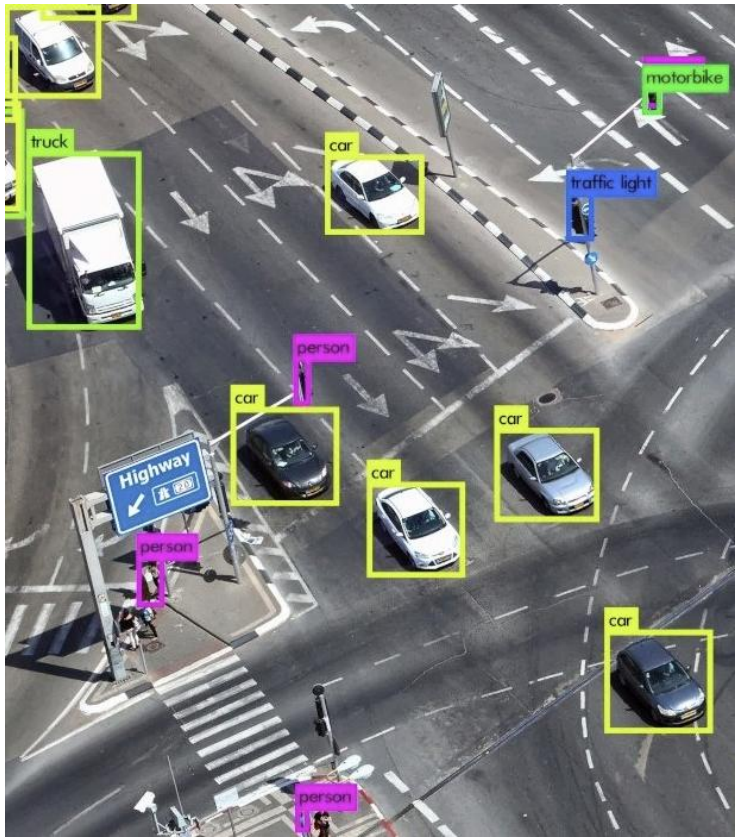


Figure 12.2 Deep network architecture with multiple layers.

Neural Network

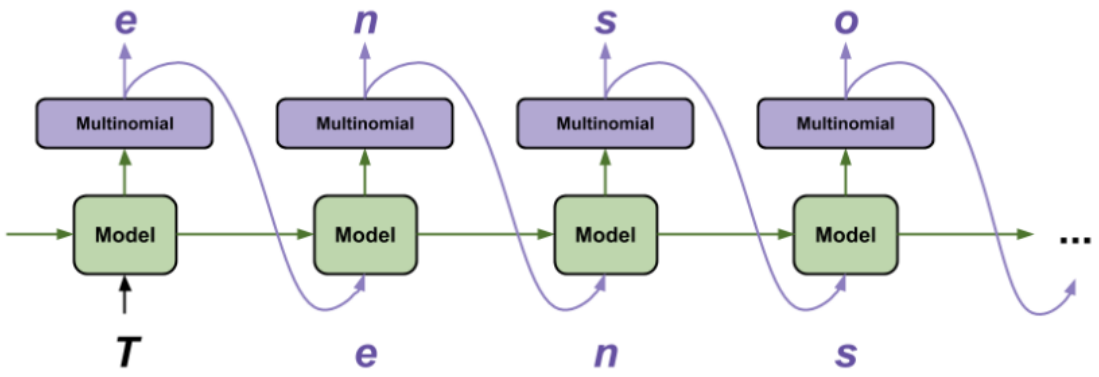
- Examples of Deep Learning
  - **Convolutional Neural Network (CNN)**
    - Image classification, object detection, video action recognition

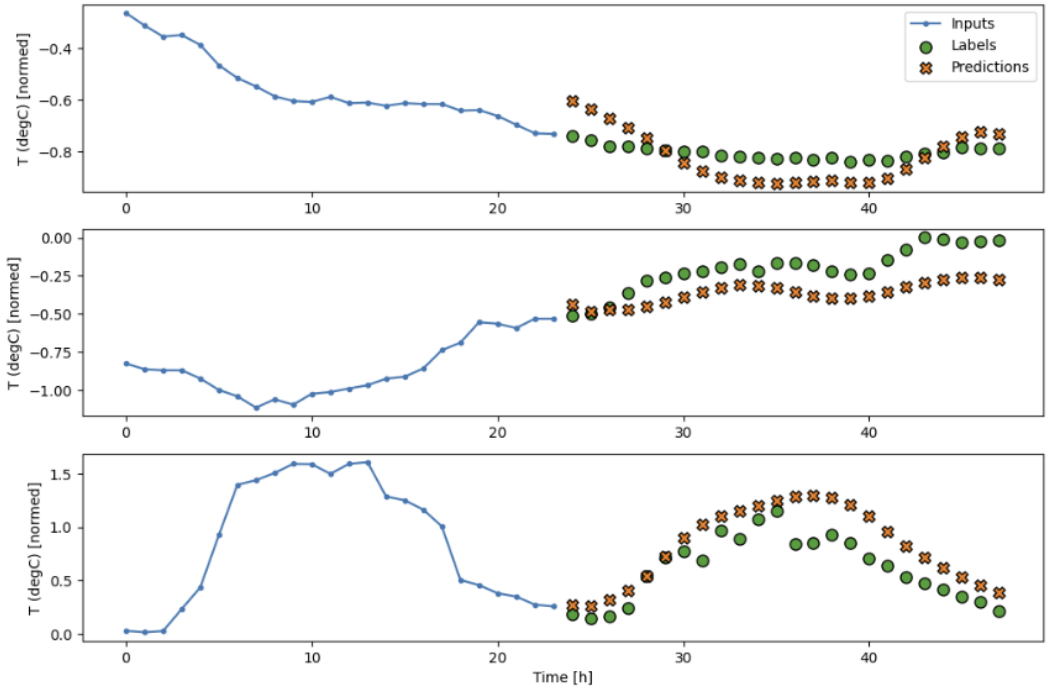




Neural Network

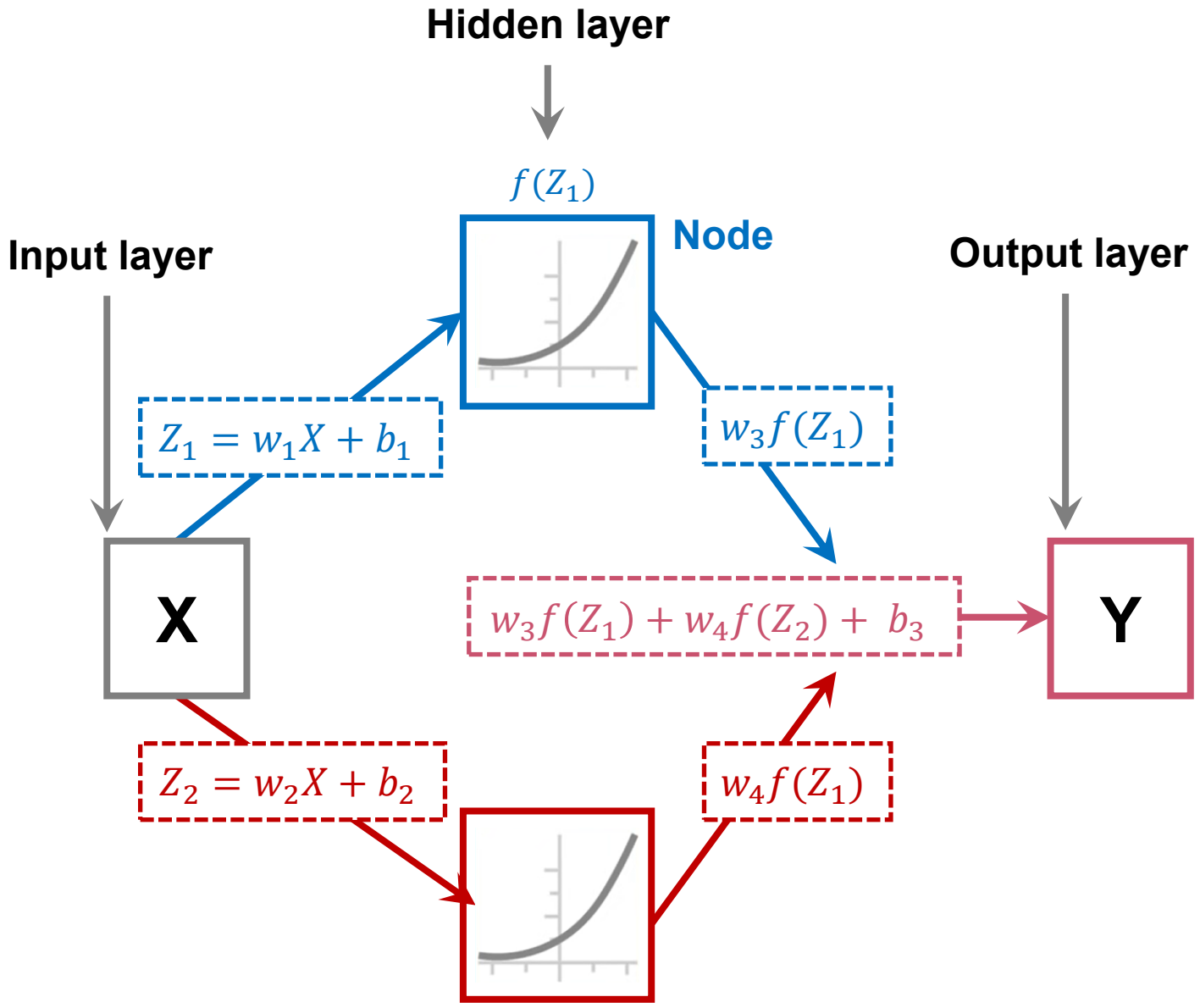
- Examples of Deep Learning
  - **Recurrent Neural Network (RNN)**
    - Text classification and generation
    - Time series forecasting





Neural Network

- **Structure**
  - **Nodes**
  - **Layers**
  - **Inputs (*X*)**
  - **Weights (*w*)**
  - **Bias (*b*)**
  - **Summation Function (*Z*)**
  - **Activation Function (*f(Z)*)**



Neural Network

- **Structure**
  - **Inputs:** Inputs are the set of values for which we need to predict an output value. They can be viewed as features or attributes in a dataset.
  - **Weights:** Weights are the real values that are attached with each input/feature and they convey the importance of that corresponding feature in predicting the final output.
  - **Bias:** Bias is used for shifting the activation function towards left or right, you can compare this to y-intercept in the line equation.
  - **Summation Function:** The work of the summation function is to bind the weights and inputs together and calculate their sum.
  - **Activation Function:** It is used to introduce non-linearity in the model.

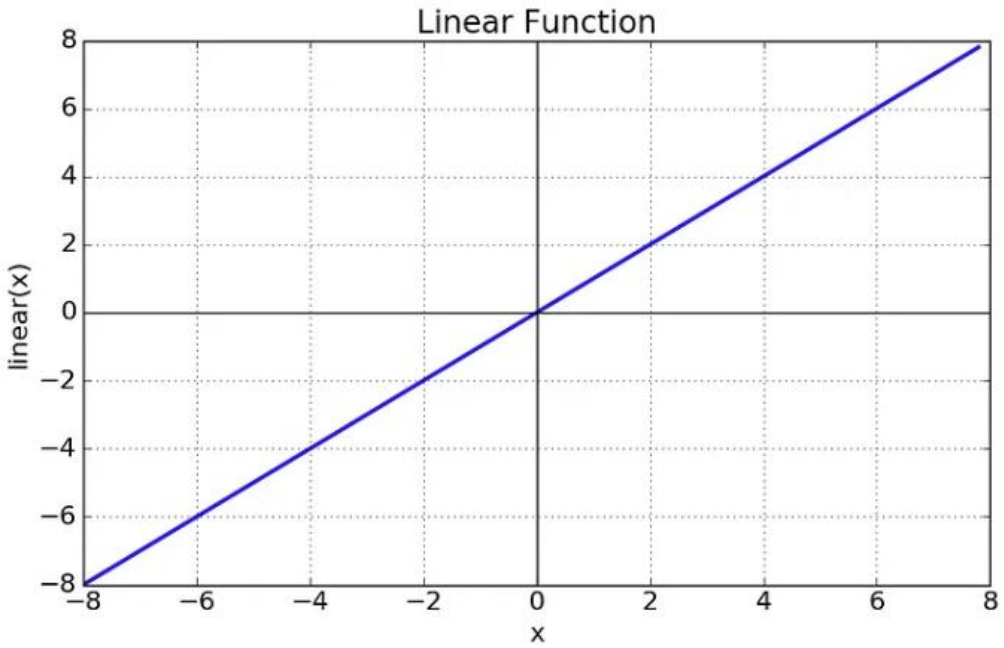
Neural Network

- **Estimation Steps**
  - Initialize the weights and biases with some random values.
  - Calculate the output which is to predict the values and estimate the loss.
    - Refer to: ***Activation function***
  - Adjust the weights and biases such that the loss will be minimized.
    - Refer to: ***Gradient descent***, ***backpropagation***

Activation Function

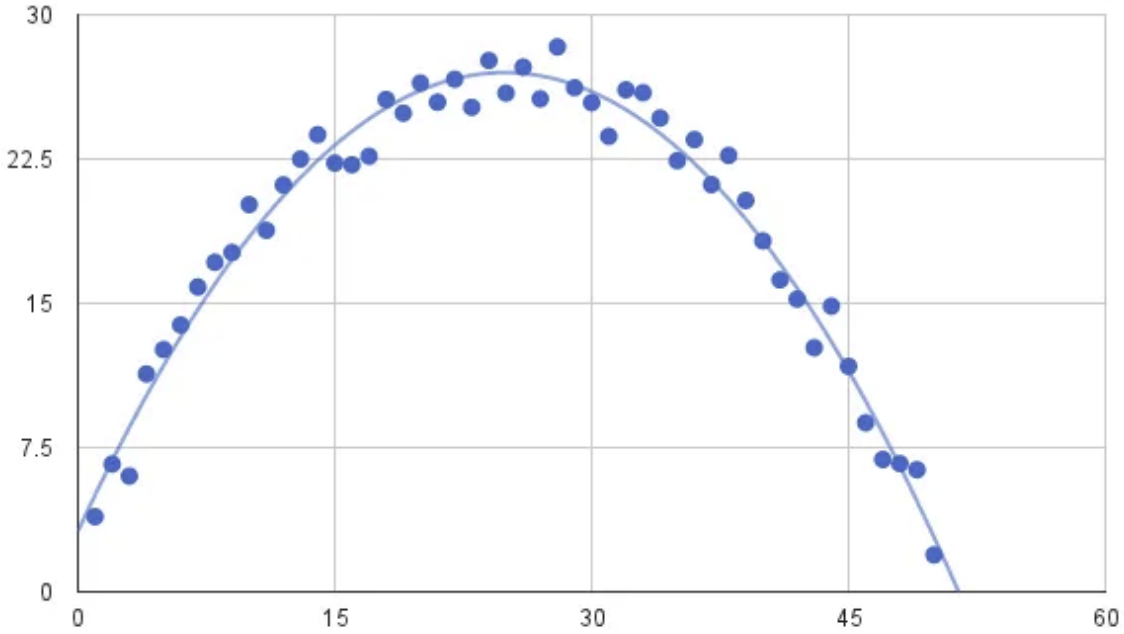
- Also known as a **transfer function**
- Mapping a linear function to another function

Linear Activation Function



Doesn't help with the complexity in neural networks

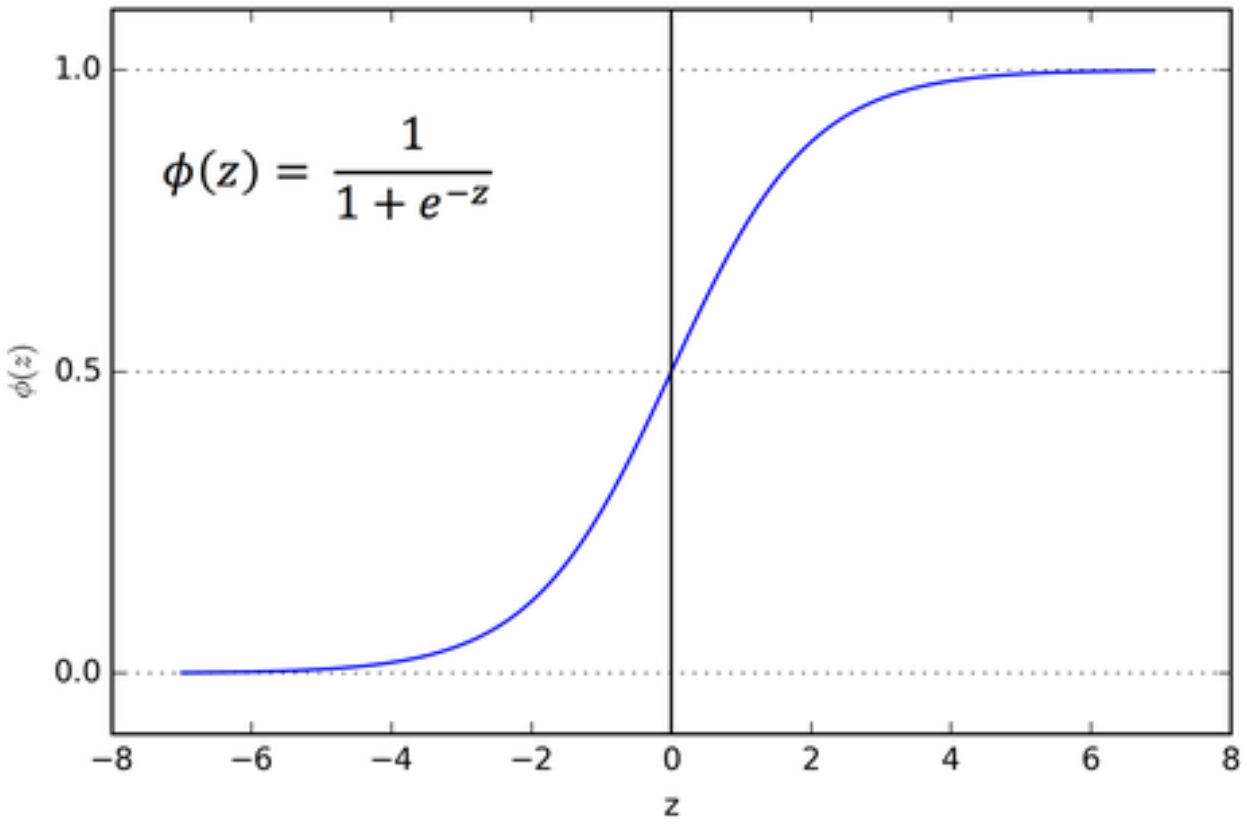
Nonlinear Activation Functions



Makes it easy for the model to adapt with variety of data

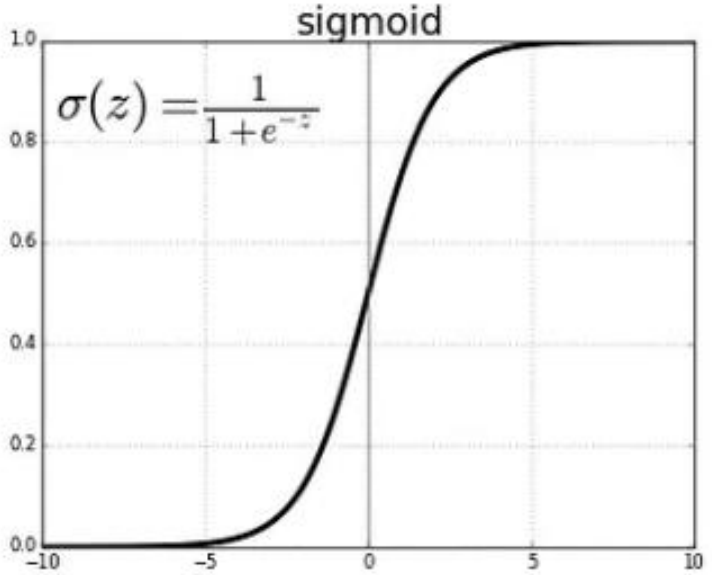
Activation Function

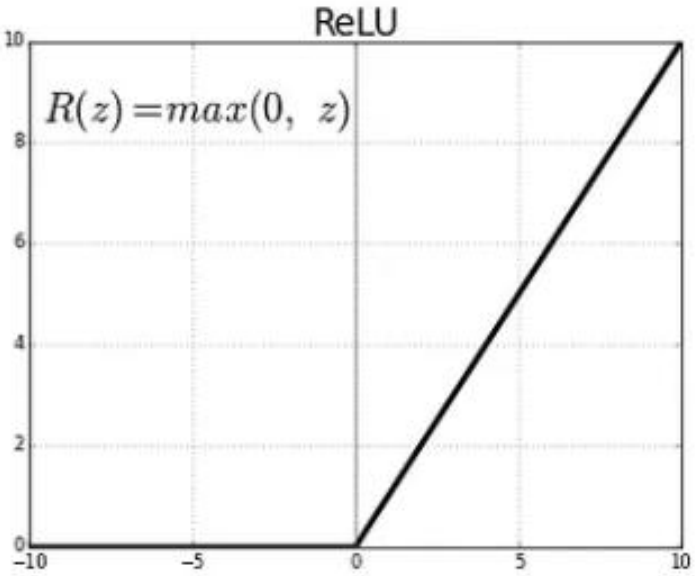
- **Sigmoid function**
  - Exists **between 0 and 1**.
  - Especially used for models to predict the probability as an output.
  - The function is **differentiable**.
  - The function is **monotonic**, but the function's derivative is not.
  - The logistic sigmoid function can cause a neural network to get stuck at the training time.
  - The softmax function is a more generalized logistic activation function which is used for multiclass classification.



Activation Function

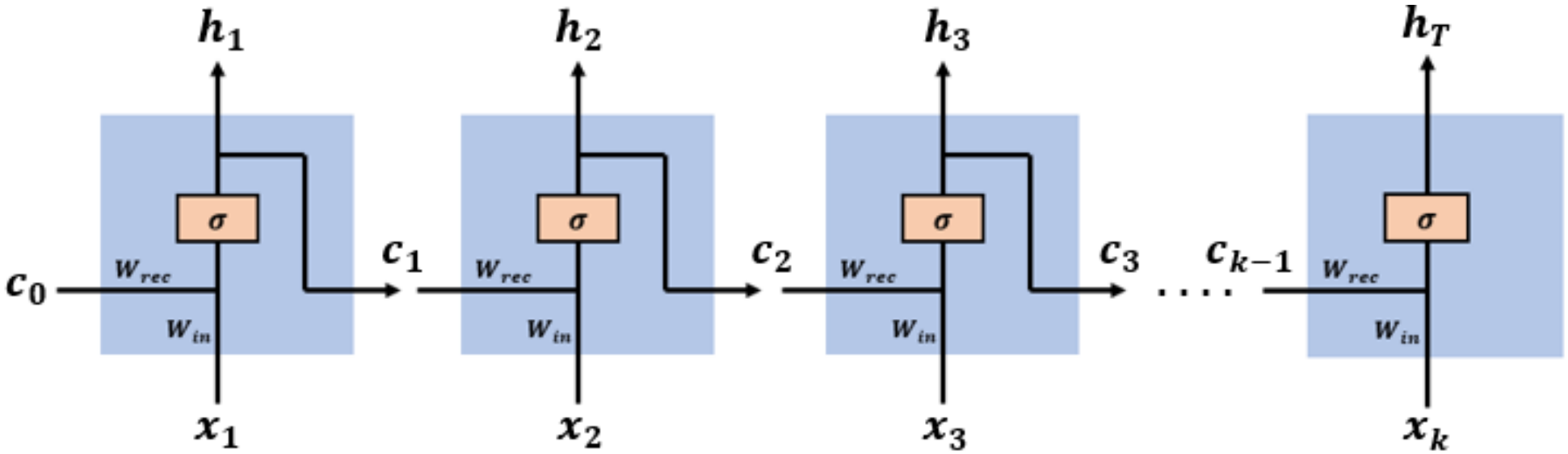
- **Rectified Linear Unit (ReLU)**
  - $R(x) = \max(0, x)$
  - Exists from 0 to infinity
  - **The most used activation function**
  - Half rectified (from bottom)
  - The function and its derivative both are monotonic.
  - All the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly.





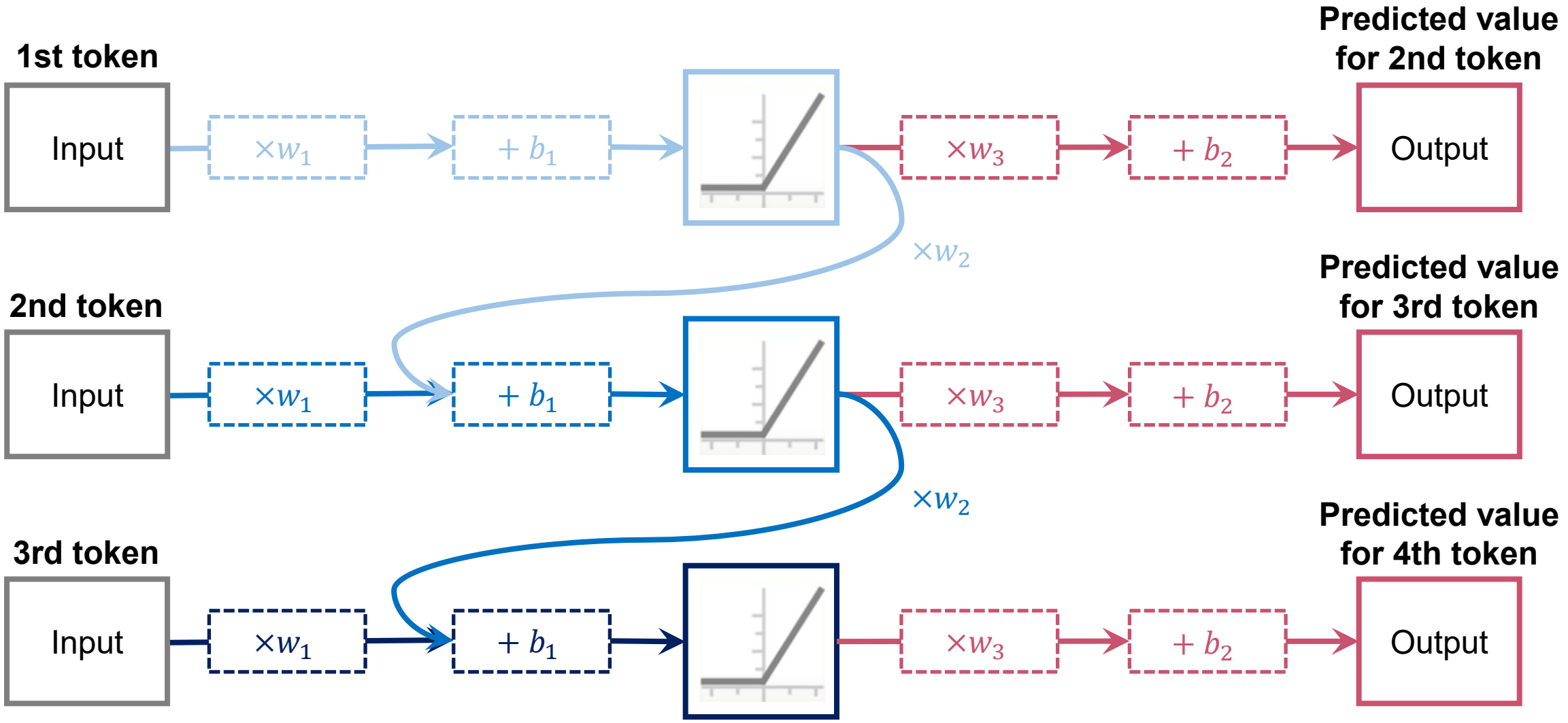
Deep Learning in Text Mining

- **Recurrent Neural Network (RNN)**
  - **Recurrent neural networks (RNNs)** are deep learning models, typically used to solve problems with sequential input data such as time series.
  - RNNs are a type of neural network that retains a ***memory*** of what it has already processed and thus can learn from previous iterations during its training.
  - An RNN is trained to recognize patterns across time, so it is suitable for handling temporal or sequential data. Thus, it is more natural to apply RNNs than CNNs, which are suitable for handling spatial data (e.g., images), although both RNNs and CNNs are applied to text classification.



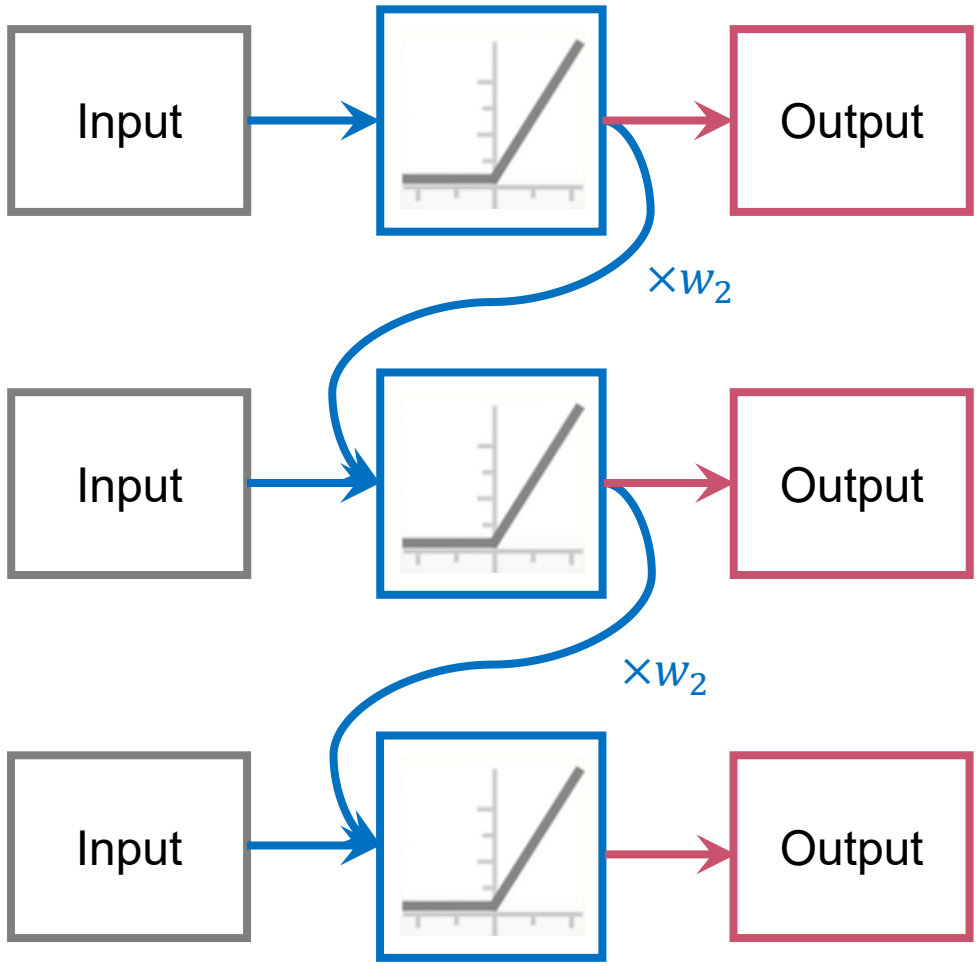
Deep Learning in Text Mining

- **Simple Illustration of RNNs**



Deep Learning in Text Mining

- **The Vanishing Gradient Problem**
  - Let's consider $w_2 < 1$ and ignore the other weights and biases.
  - The influence of input after n time points becomes w_2^n times.
  - For instance, if we consider $w_2 = 0.5$ and $n = 20$, then $w_2^n = 0.5^{20} < 0.000001$.
  - If we try to train this RNN with backpropagation, this number will lead to an extremely small gradient.

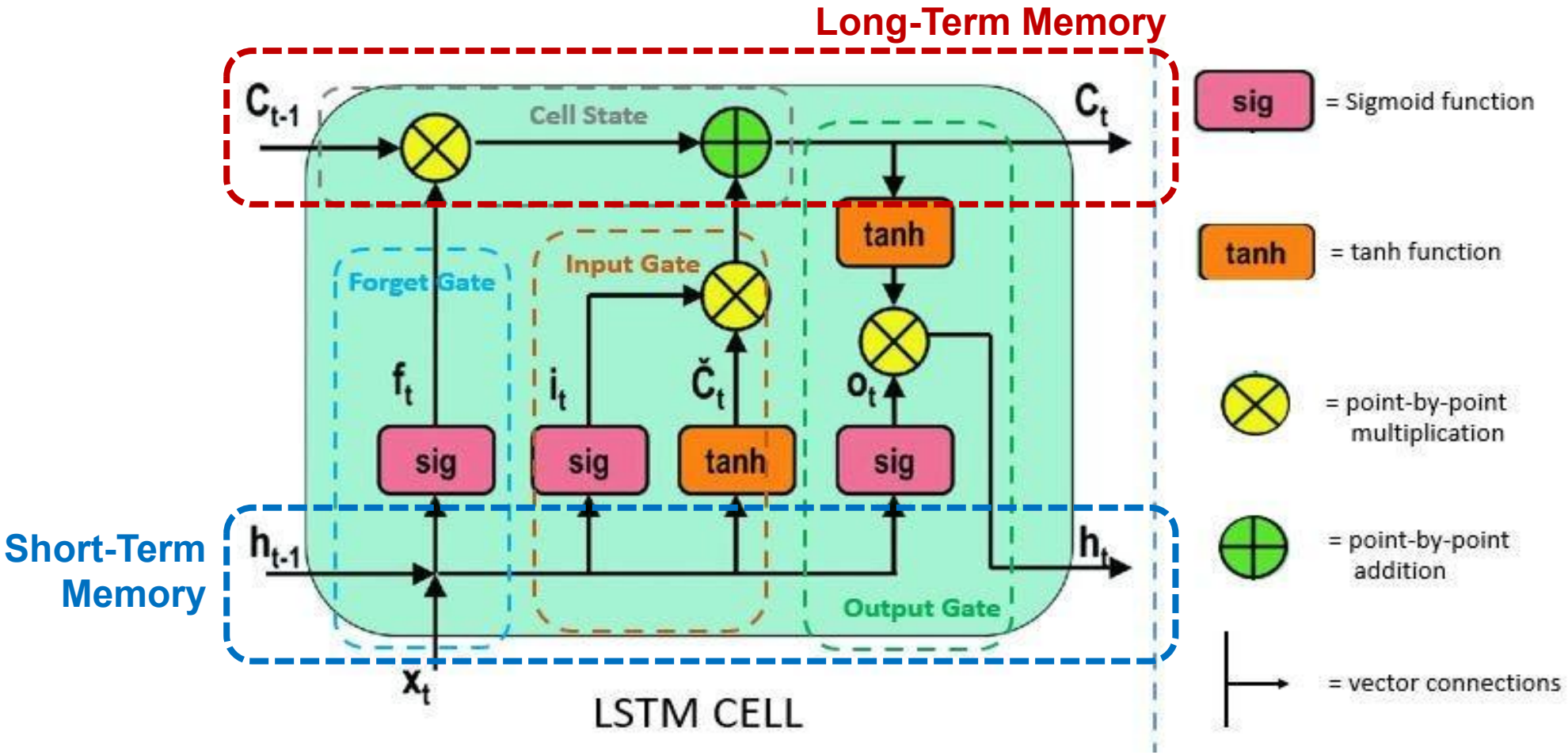


Deep Learning in Text Mining

- **Long Short-Term Memory (LSTM)**
  - **LSTM** is an extension of earlier RNNs, capable to retain ***a long-term memory*** and use it to learn patterns in longer sequences of source data.
  - Prior to LSTMs, RNNs were forgetful. They could retain a memory, but only about the process steps in their immediate past (***vanishing gradient problem***).
  - The LSTM, by contrast, introduces loops that can generate long-duration gradients. At each time step, it can tie together three pieces of information: the ***current input data***, the short-term memory it receives from the preceding cell (the so-called ***hidden state***), and the long-term memory from more remote cells (the so-called ***cell state***), from which the RNN cell produces a new hidden state.

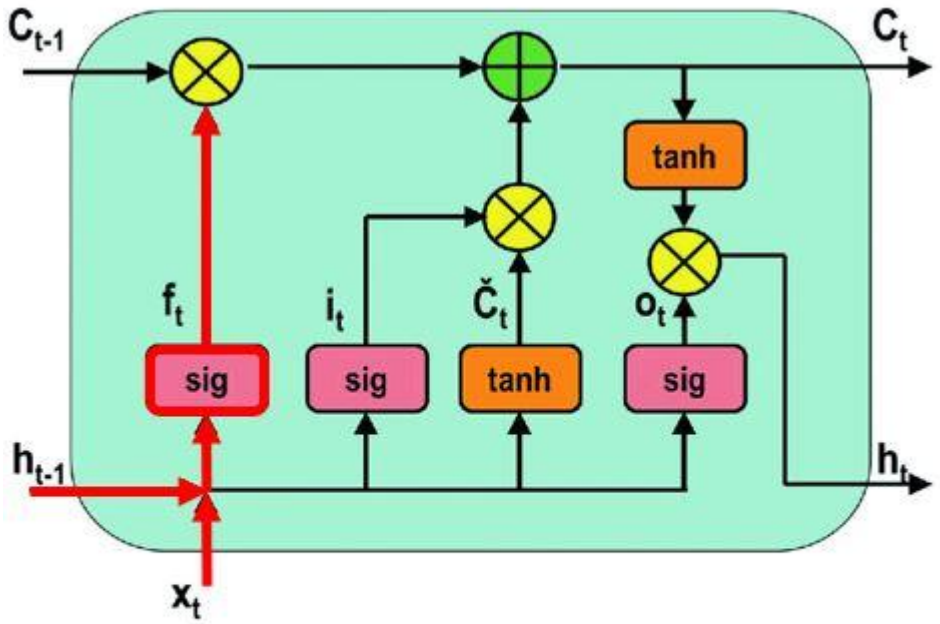
Deep Learning in Text Mining

- **Structure of LSTM**



Deep Learning in Text Mining

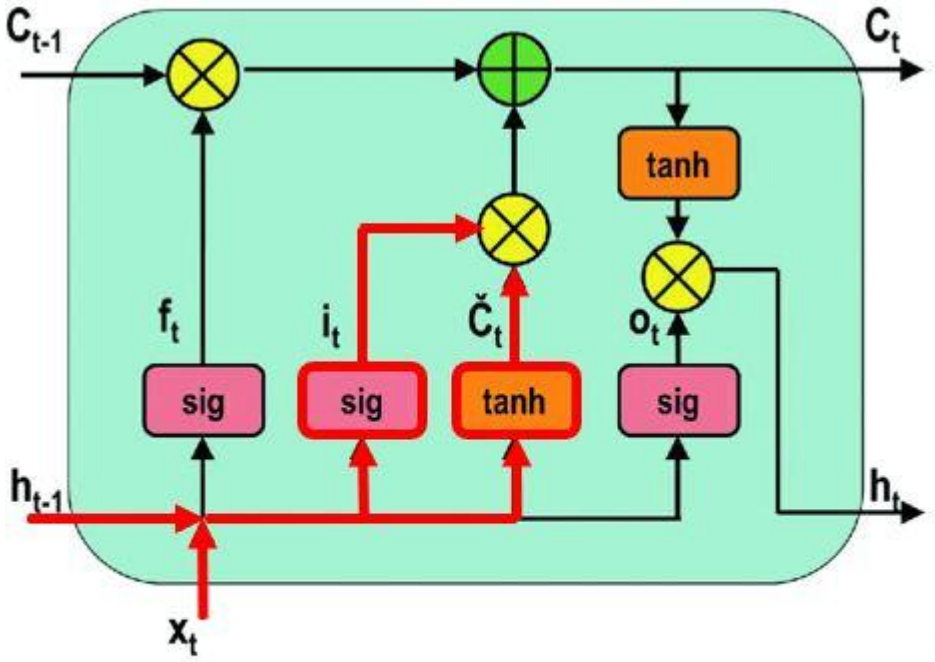
- **Structure of LSTM: Forget Gate**
  - This gate decides what information should be carried out forward or what information should be ignored.
  - Information from previous hidden states and the current state information passes through the sigmoid function. Values that come out from sigmoid are always between 0 and 1. if the value is closer to 1 means information should proceed forward and if value closer to 0 means information should be ignored.



Forget Gate Operation

Deep Learning in Text Mining

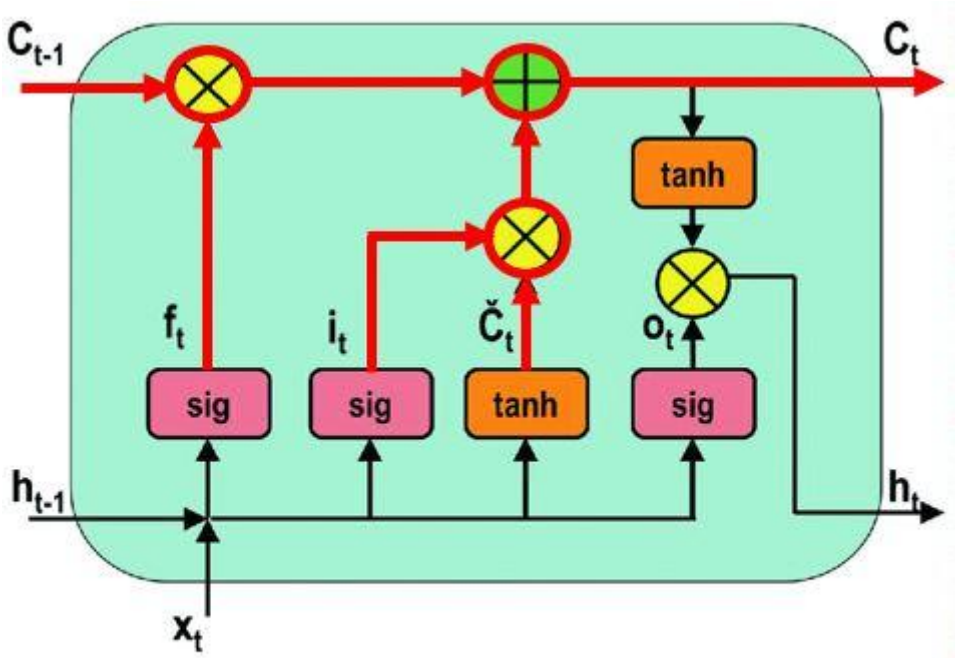
- **Structure of LSTM: Input Gate**
  - After deciding the relevant information, the information goes to the input gate, Input gate passes the relevant information, and this leads to updating the cell states.
  - Input gate adds the new relevant information to the existing information by updating cell states.



Input Gate Operation

Deep Learning in Text Mining

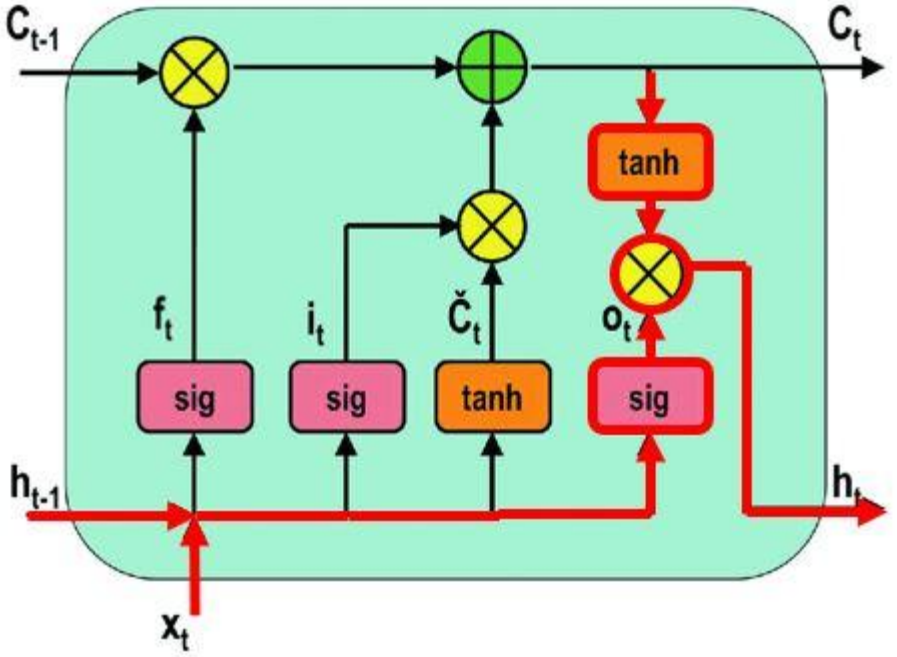
- **Structure of LSTM: Cell State**
  - After the network gains information from the forget gate and input gate, LSTM decides and stores the information from the new state in the cell state.
  - The **previous cell state** C(t-1) gets multiplied with **forget vector** f(t).
  - Next, the network takes the output value of the **input vector** i(t) and performs point-by-point addition, which updates the cell state giving the network **a new cell state** C(t).



Cell State Operation

Deep Learning in Text Mining

- **Structure of LSTM: Output Gate**
  - After the information is passed through the input gate, now the output gate comes into play.
  - Output gate generates the next hidden states, and cell states are carried over the next time step.



Output Gate Operation

Live Quiz

- **Join the quiz!**
  - https://pollev.com/jaeungsim088



Transformer

- **Transformer (Vaswani et al. 2017)**



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Attention is all you need

A Vaswani, N Shazeer, N Parmar… - Advances in neural …, 2017 - proceedings.neurips.cc

… to attend to **all** positions in the decoder up to and including that position. **We need** to prevent … **We** implement this inside of scaled dot-product **attention** by masking out (setting to −∞) …

☆ Save 99 Cite Cited by 240098 Related articles All 71 versions »

Transformer

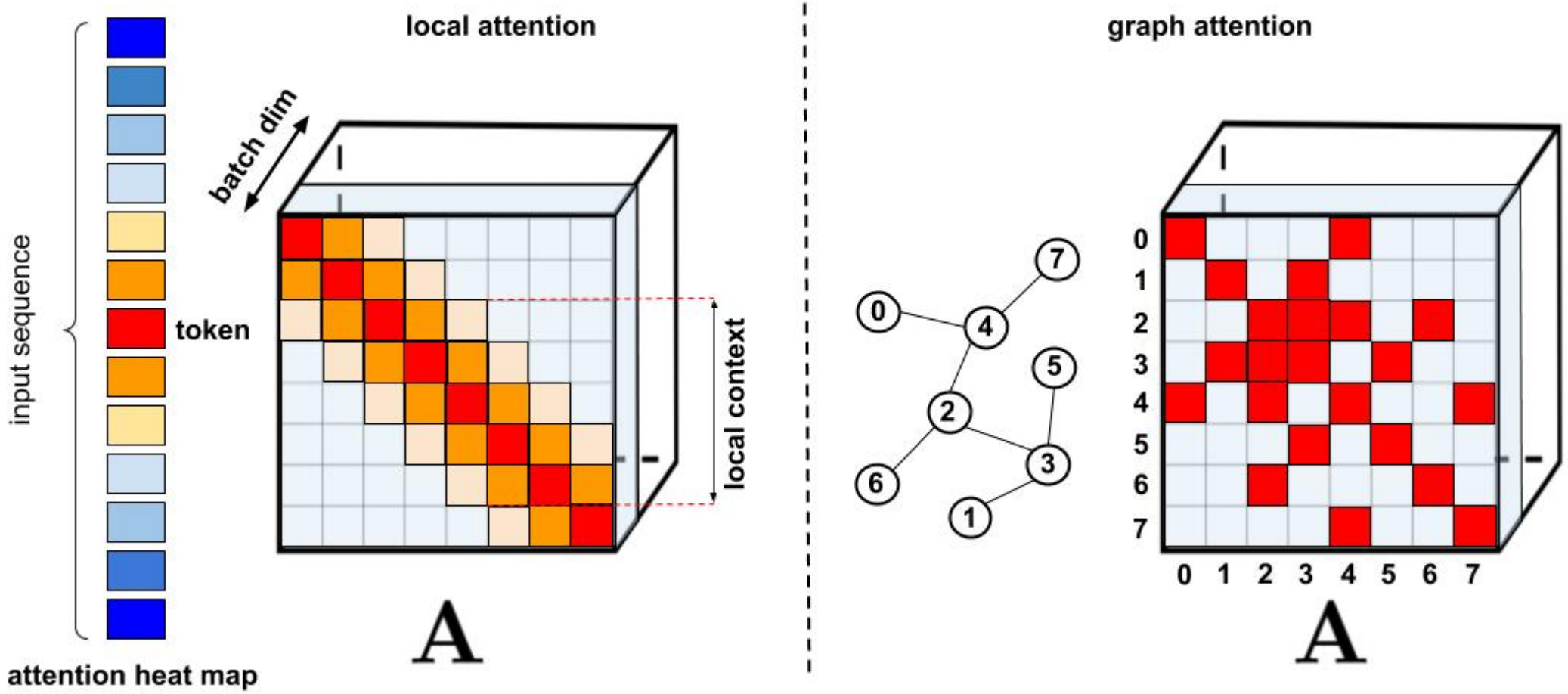
- **Examples**
  - BERT…stands for?
  - GPT…stands for?
  - Gemini
  - Claude
  - Grok
  - Llama stands for?

Transformer

- **Attention is all you need?**
  - Each word decides how much attention to give to every other word.
  - Sentence:
    - “The bank approved the loan because it was profitable.”
  - What does “it” refer to?
    - “bank”?
    - “loan”?
  - A Transformer assigns attention weights and figures out:
    - “it” → “loan”

Transformer

- **Attention is all you need?**



Source: https://research.google/blog/rethinking-attention-with-performers/

Transformer

- **Transformer (Vaswani et al. 2017)**
  - **Encoder-Decoder Structure**
    - The Transformer is composed of an Encoder to process the input, and a Decoder to generate the output.
    - Each of these is made up of a stack of identical layers containing self-attention and feed-forward neural networks.

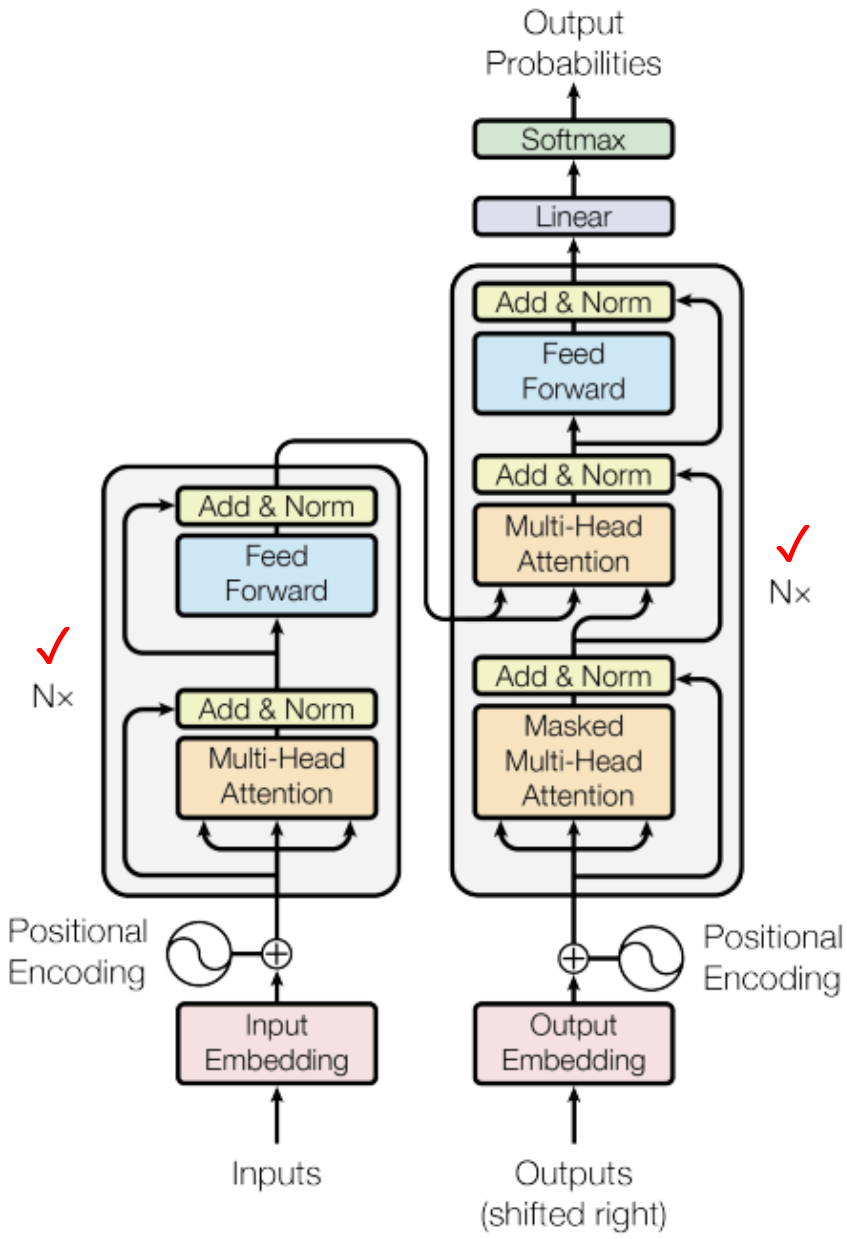


Figure 1: The Transformer - model architecture.

Transformer

- **Transformer (Vaswani et al. 2017)**
  - **Positional Encoding**
    - Since Transformers don't inherently understand the order of tokens, positional encoding is added to the input embeddings to give the model information about the position of a word in a sentence.

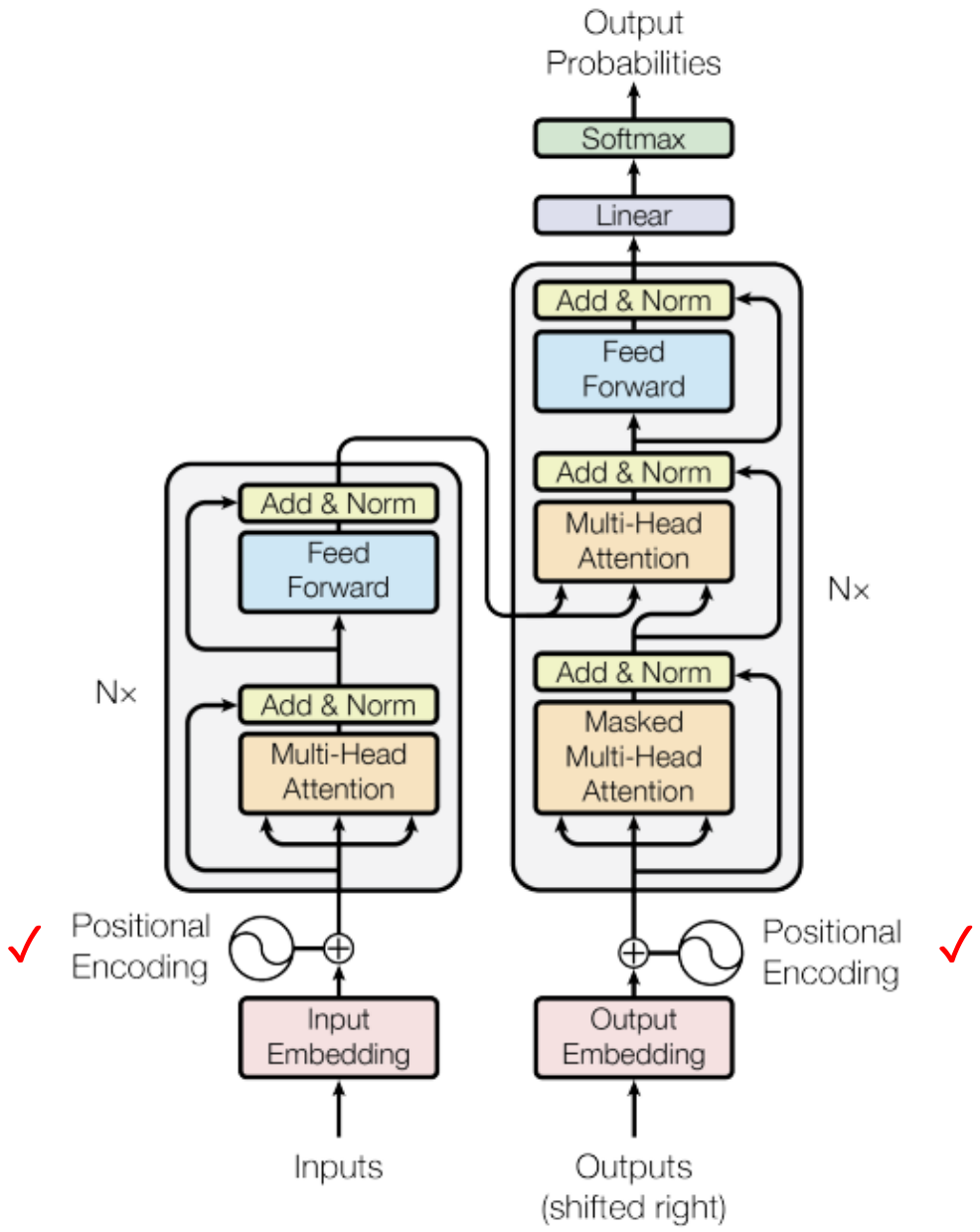


Figure 1: The Transformer - model architecture.

Transformer

- **Transformer (Vaswani et al. 2017)**
  - **Self-Attention Mechanism**
    - Transformers use a mechanism called self-attention, enabling each element in the input sequence to "attend" to every other element in the sequence, thereby capturing dependencies regardless of distance in the input sequence.
  - **Multi-Head Attention**
    - Instead of having a single set of attention weights, the model has multiple sets (or "heads"), allowing it to focus on different parts of the input sequence when producing each element in the output sequence.

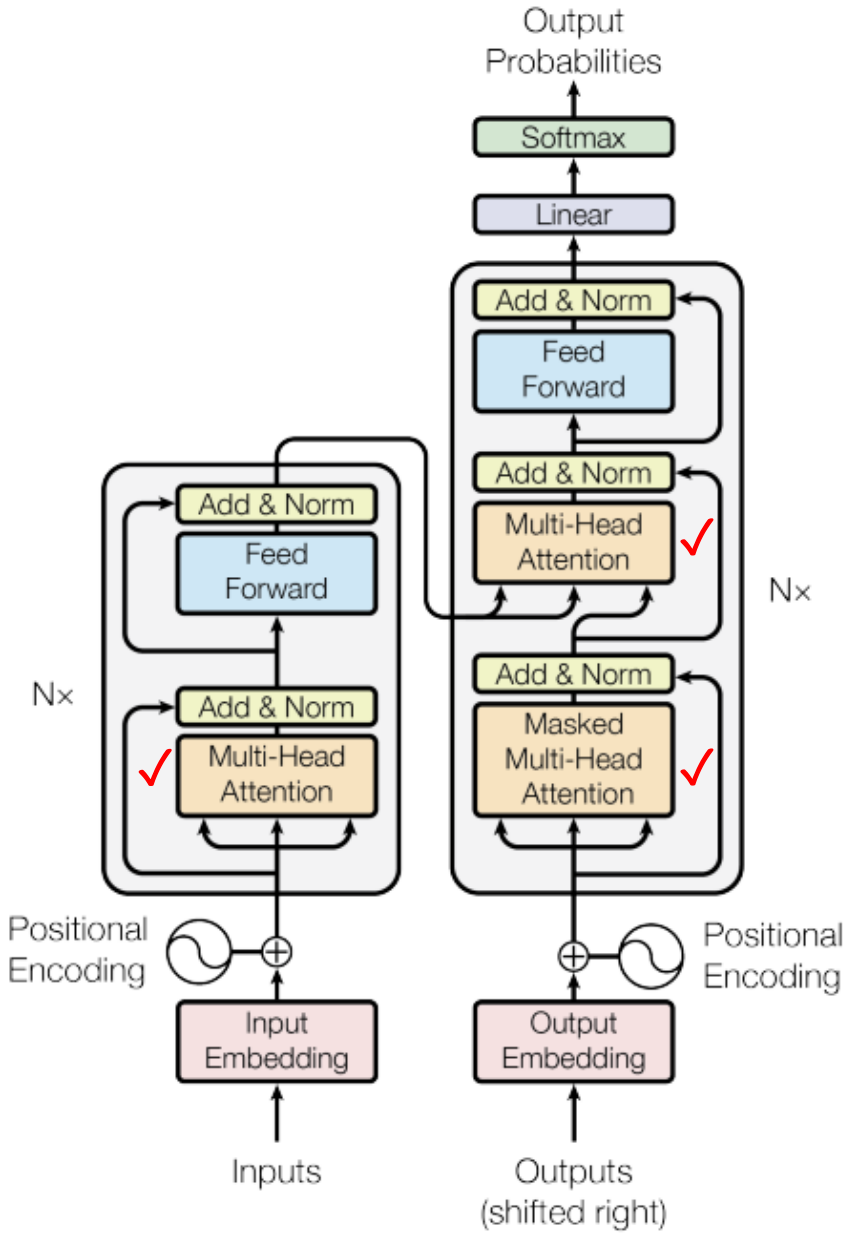


Figure 1: The Transformer - model architecture.

Transformer

- **Transformer (Vaswani et al. 2017)**
  - **Layer Normalization**
    - Each sub-layer (like self-attention, feed-forward neural network) has a residual connection around it followed by layer normalization.
  - **Feed-Forward Networks**
    - The feed-forward network is present in both encoder and decoder, and it is applied independently to each position.

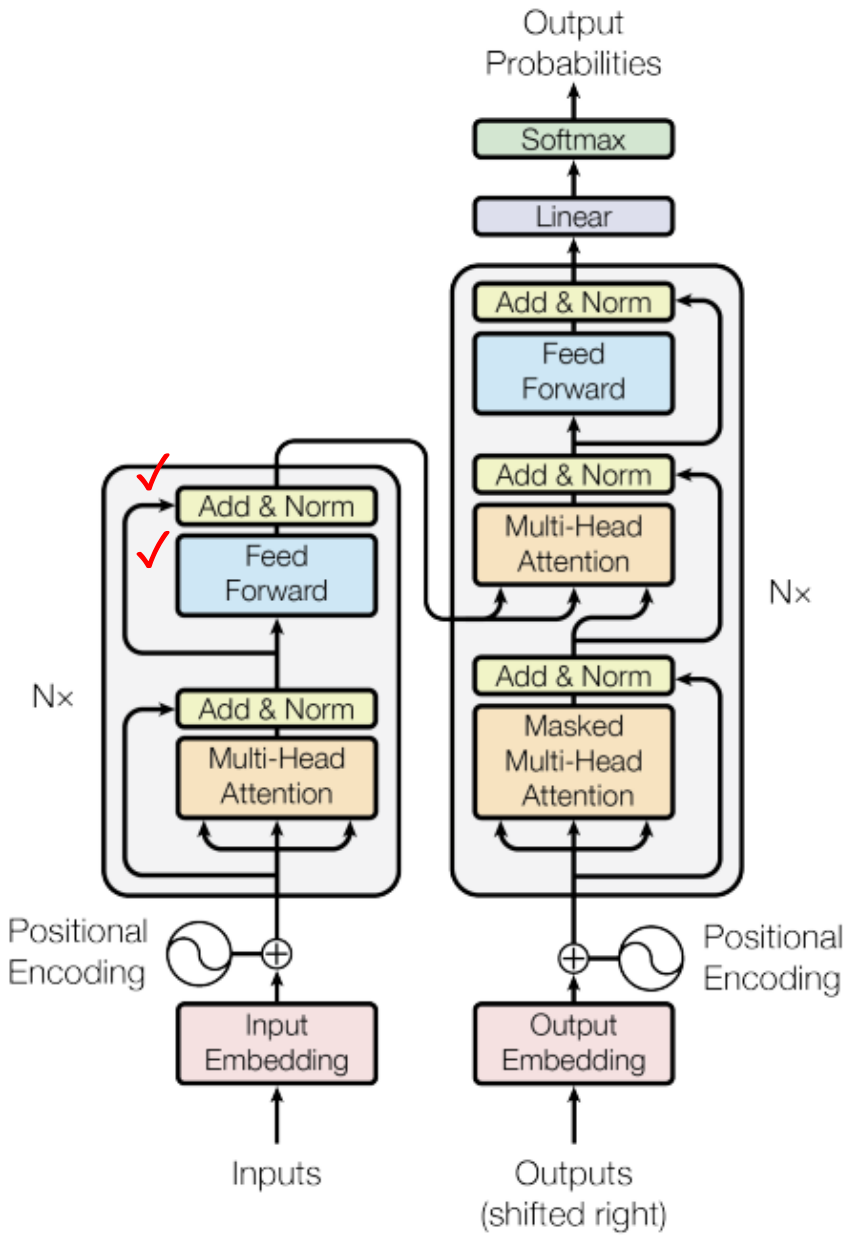


Figure 1: The Transformer - model architecture.

Transformer

- **Transformer (Vaswani et al. 2017)**
  - **Final Linear and Softmax Layer**
    - The Decoder's output is transformed to the output vocabulary size using a linear layer and softmax is applied to predict the next word in the sequence.

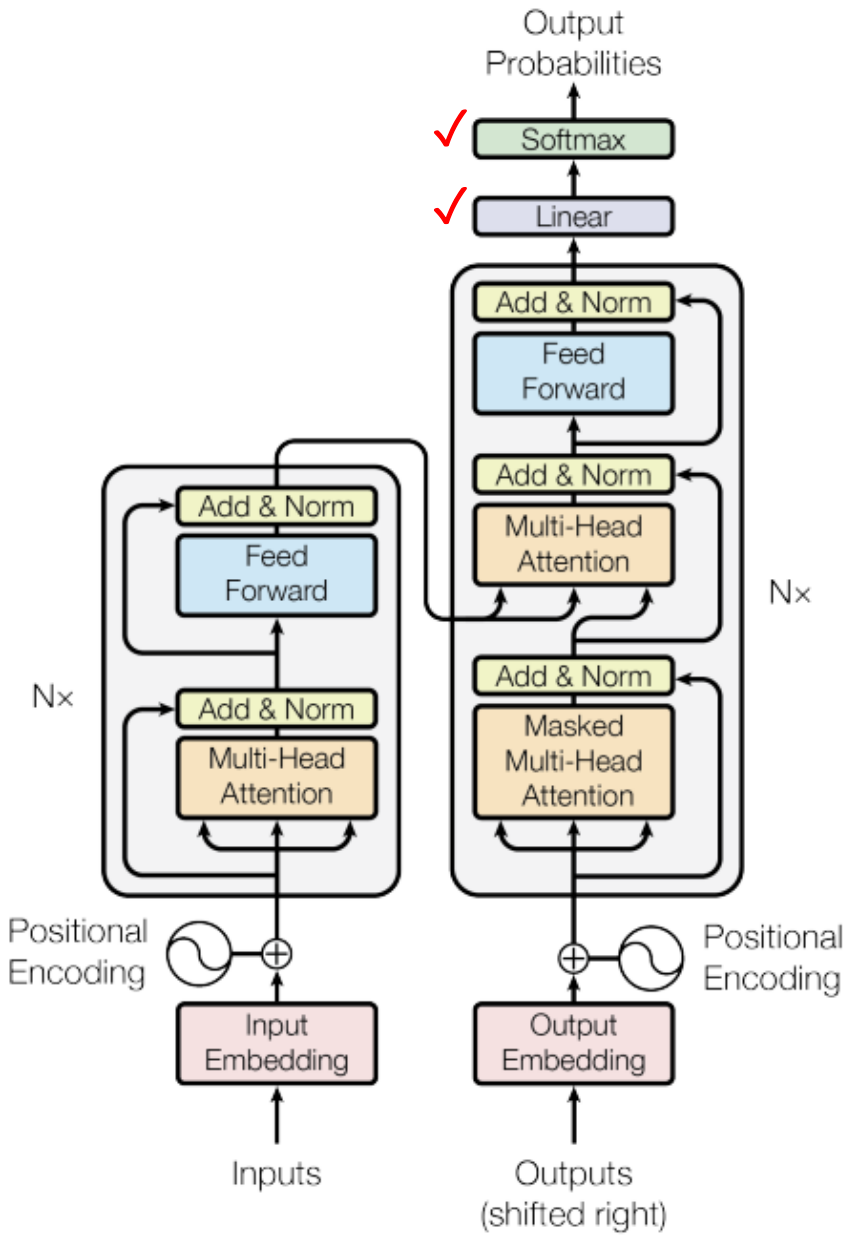


Figure 1: The Transformer - model architecture.

Transformer

- **Major Advantages**
  - **Parallelization**
    - Unlike RNNs, Transformer models do not require sequential data processing, enabling high levels of parallelization during training and thereby reducing training times significantly.
  - **Long-Range Dependencies**
    - Transformers use self-attention mechanisms, which allow them to weigh and consider all tokens in the input sequence, regardless of their distance, effectively capturing long-range dependencies in the data.
  - **Scalability**
    - The architecture is highly scalable, meaning it can accommodate increasing amounts of data and compute resources efficiently. This characteristic has led to the development of very large models like GPT-3 and GPT-4.

Transformer

- **Major Advantages (cont'd)**
  - **Versatility**
    - Transformers have demonstrated state-of-the-art performance across a wide range of NLP tasks, including but not limited to machine translation, text summarization, question answering, and text classification.
  - **Pre-training and Fine-tuning**
    - Transformers can be pre-trained on large corpora of unlabeled data and fine-tuned on smaller, task-specific datasets, which allows them to generalize well even when labeled data is scarce.

Transformer

- **Transformer (Vaswani et al. 2017)**
  - Comprehensive explanations are available in various sources:

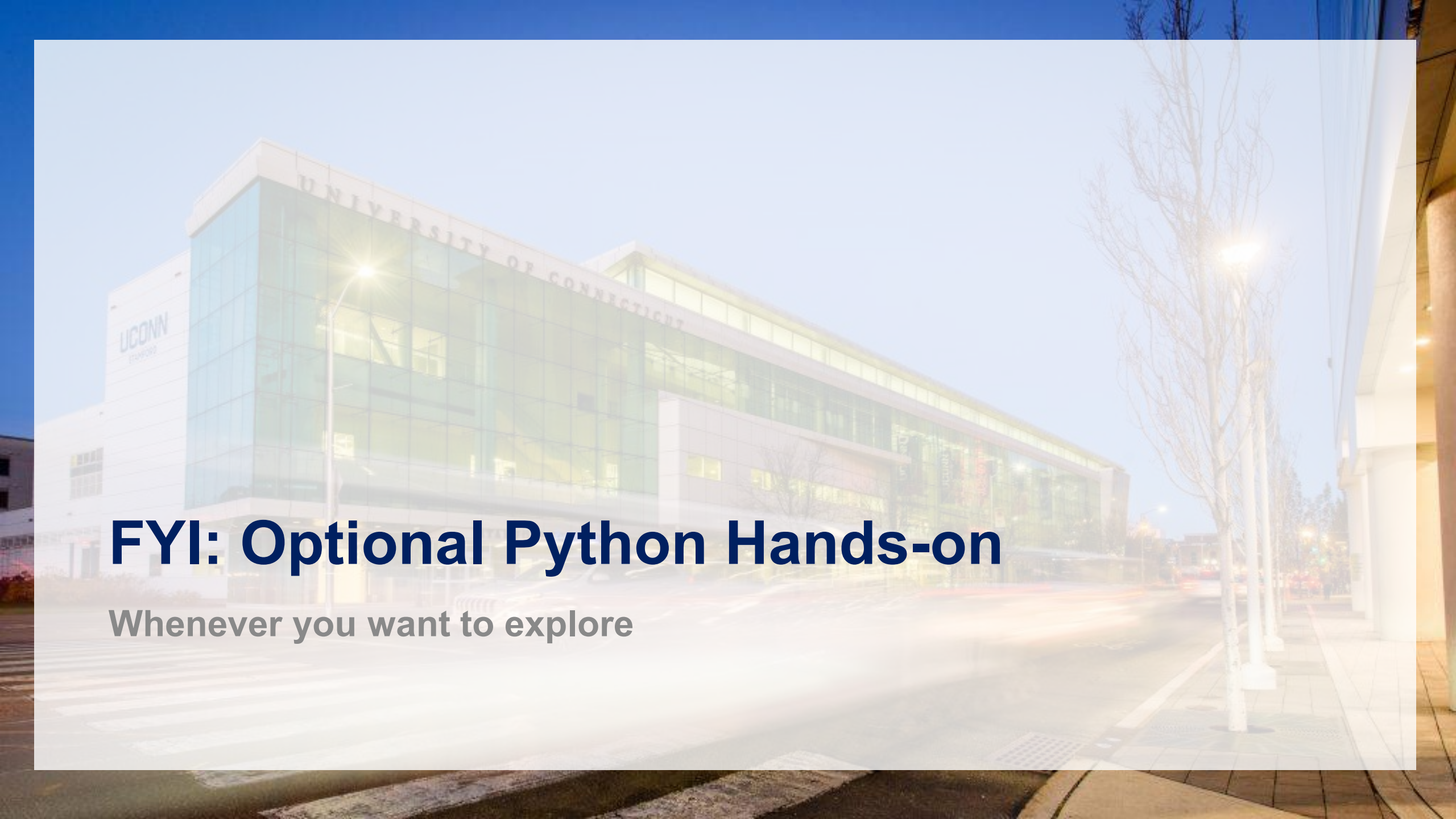


Source: https://www.youtube.com/watch?v=4Bdc55j80l8

Live Quiz

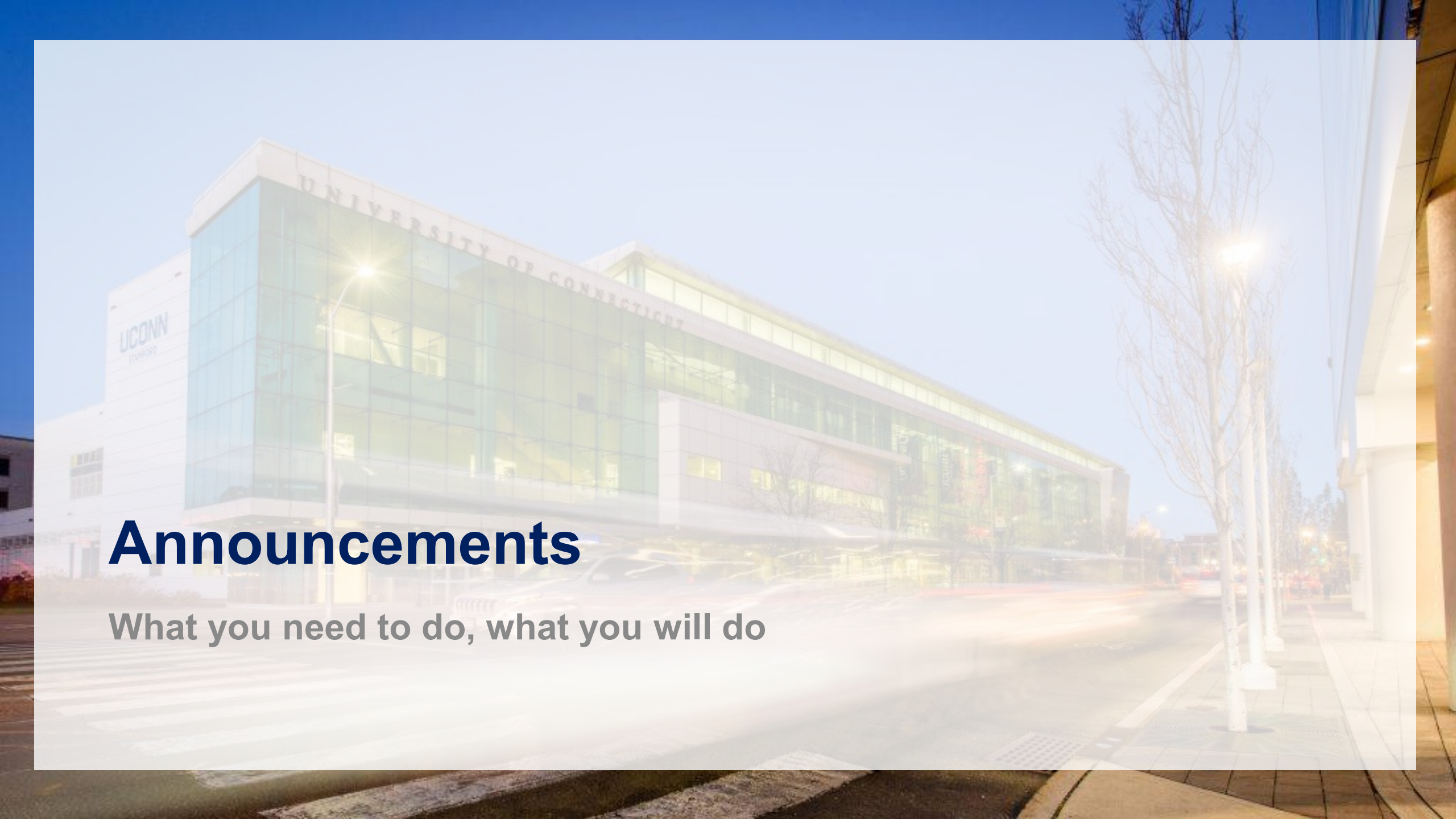
- **Join the quiz!**
  - https://pollev.com/jaeungsim088





FYI: Optional Python Hands-on

Whenever you want to explore



Announcements

What you need to do, what you will do

Announcements

- **Nov 8 (Sun), 2026: Homework #2 Deadline**
- **Nov 10 (Tue), 2026: Recent Challenges in Machine Learning**
- **Nov 17 (Tue), 2026 (Virtual): Term Project Presentation #2**
- **Nov 24 (Tue), 2026 (No Class): *Thanksgiving Recess***
- **Dec 1 (Tue), 2026 (Asynchronous): Take-home final exam will be released.**



Jaeung Sim

Assistant Professor

School of Business, University of Connecticut

jaeung.sim@uconn.edu